Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	**Master of Computer Application**	
Course	**MJ Programming from First Principles**	
Topic Name	**A means to understand the built-in types in a uniform framework (practice session), part 2.2**	
Topic Code	**-**	
Unit/Module No. & Name	**10**	**User-defined types**
Self-Learning Hours	**-**	

Topic Details

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited

OBJECTIVE

1. The main goal of this unit is to help you understand user-defined types as tools for modeling real-world ideas inside a program.
2. A model is a small, clear picture of a part of the real world, and in programming this picture is created using data types and code.
3. User-defined types let you draw this picture in an organized way by creating types like Student, Track, Artist, or Appointment instead of using only numbers and strings.
4. The unit also explains that user-defined types extend a programming language without changing the language itself.
5. You do not rewrite the compiler; instead, you design your own family of types that act like new “words” in your program’s language.
6. In this way, each program becomes a s
7. mall language shaped by the user-defined types inside it.
8. After completing this unit, you should be able to explain in simple words what a data type and a user-defined type are.
9. You should be able to read a short story about a system (like a music app or hospital system) and identify important nouns that can become user-defined types, such as Track, Artist, Patient, or Appointment.
10. You should be able to list basic fields for these types, such as name, id, or durationInSeconds.
11. You should be able to explain the difference between built-in types and user-defined types and give reasons why both are needed.
12. You should be able to describe common forms of user-defined types, such as records/structs, classes/objects, and enumerations.
13. You should be able to follow and explain simple case studies where these types work together to model a small but realistic system.

1.0 INTRODUCTION

1.1 Why We Study Data Types

Every computer program works with some kind of data. A calculator uses numbers. A chat app uses messages, usernames, and time stamps. A music player uses songs, artists, and playlists. When you write a program, you must tell the computer what type of data you want to store and what you want to do with it. This is the job of a data type. Without types, the computer would not know how to store values or what actions are allowed on them.

Most programming languages give you basic types like integer, float, character, string, and Boolean. These are like small building blocks. They are helpful but not enough to represent real-life things. For example, a student is not just a name — a student has an ID, courses, grades, and contact details. To keep all these details together, we need a user-defined type.

User-defined types let you create your own custom blocks. You can group related information together and give it a name. The computer then treats this group as a new type, just like int or string. In this way, you act like a small language designer inside your own program.

As programs grow, handling many separate variables becomes confusing. Some variables always come together in pairs or groups. At this point, user-defined types become very important. They give clear structure to your code and make the purpose of the program easier to understand for both the computer and the programmer.

1.1.1 Real World And Programs

There is a strong connection between the real world and the world inside a computer program. When we talk about modeling, we mean creating a simple picture of real things in a form a computer can use. A map is a simple model of a city. In the same way, a user-defined type is a model of something real (or imaginary) inside a program. The goal is not to copy the whole world, but to keep only the details needed for the task.

You cannot store a real person in a computer, but you can store important information about the person. You choose which details matter and how to arrange them. If this is done well, the type becomes simple for the computer and useful for the program. Good modeling means knowing what to include and what to leave out.

For an MP3 track, the real sound is stored in a file, but the program keeps details like the song title, artist name, album name, track length, and file location. A user-defined type called Track is a clean way to store all these details together. This helps the music player search, sort, and play songs easily and safely.

2.0 MEANING AND DEFINITION

2.1 Meaning Of Data Type

In computer science, a data type is a rule that tells us what kind of value we have and what operations we can perform on that value. An integer type allows addition and subtraction. A string type allows operations like joining two strings or searching for a letter. A Boolean type allows logical operations such as “and,” “or,” and “not.” The type helps the compiler or interpreter check that you are using values in a safe and meaningful way.

We can think of a type as a label plus a set of allowed actions. When we declare a variable with a type, we are saying, “This box will hold values of this shape, and I will use them in these ways.” If we try to use the variable in a wrong way, the type system can warn us. In this sense, the type system works like an assistant that looks over your code and points out likely mistakes before the program runs or while it runs.

2.1.1 Meaning Of User Defined Type

A user defined type is a data type that is created by the programmer, using language features such as structures, classes, records, or enumerations. While primitive types are supplied by the language designers, user defined types come from the needs of a specific problem. Whenever the simple types are not enough to express an idea clearly, the programmer can invent a new type that fits that idea.

We define a user defined type by giving it a name and describing its internal structure. The structure can be made from primitive types and from other user defined types. Once we have defined such a type, we can declare variables of that type, pass them to functions, return them from methods, and store them in collections. In this way, the new type becomes a full citizen of the language, almost as if the language had always had it.

In an MP3 player, a Track type might have fields like title, artist, album, durationInSeconds, and filePath. Taken together, these fields form one value that stands for “one MP3 track” inside the program. The music library then works with Track values instead of loose strings and numbers.

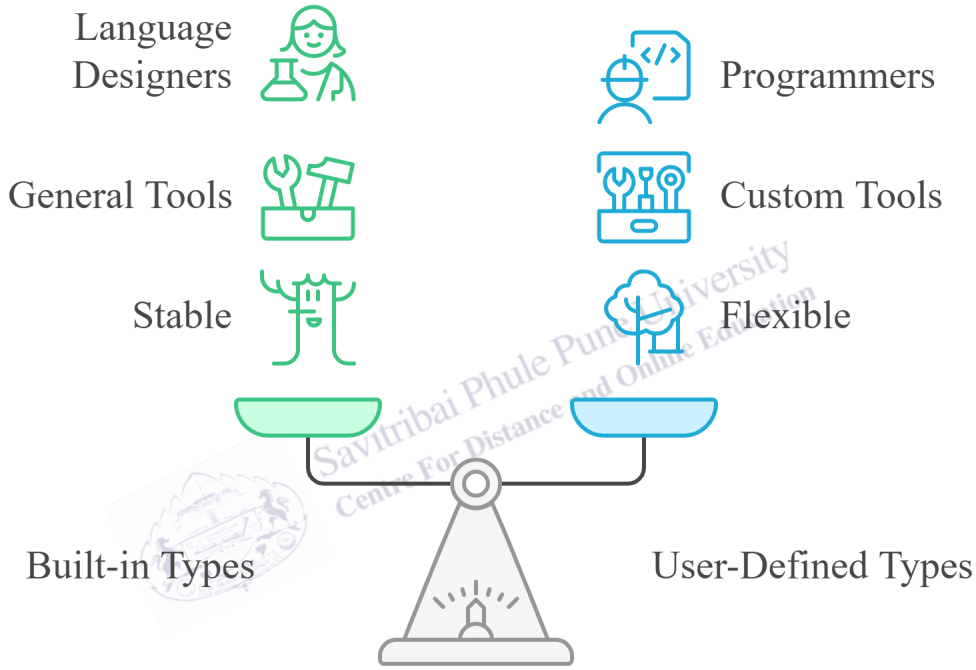
2.2 Formal View Of User Defined Types

In a more formal view, a user defined type describes a set of possible values and a set of operations that make sense for those values. The values are built from other types using builders such as “record of,” “list of,” or “either this or that.” The operations may include reading and writing fields, comparing two values, or applying domain specific rules, such as checking that a track duration is positive or that an appointment date is not in the past.

Formal thinking about types helps when we reason about program correctness. If we know that a certain type always has a nonempty list of items, we can prove that some functions never face an empty input. If we know that a field is an enumeration with only three states, we can check that our code handles all three states and does not forget a case. In this way, user defined types are more than containers; they are precise descriptions that support proofs and careful arguments about code.

Fig. 1

Balancing Built-in and User-Defined Types



This diagram explains the balance between built-in and user-defined data types. Built-in types are designed by language creators and are stable and general, while user-defined types are created by programmers to build flexible, customized solutions for specific problems.

2.2.1 Built-In and User-Defined Types

Built in types are general tools that work in almost every program. They are few in number, stable, and highly tuned by the language implementers. Examples include integers, floating point numbers, characters, strings, and Booleans. User defined types are custom tools that match one family of problems well. They can be many, flexible, and ready to change as the design of the system changes.

In practice, we combine both. Primitive types handle low level details such as counts, indices, and simple flags. User defined types handle higher level ideas such as Customer, Order, Playlist, or Appointment. In our MP3 model, integers are still used for things like duration in seconds and index positions, but the main story is told in terms of Track, Artist, Album, and Playlist types. This layered use of types leads to clearer and safer designs.

2.2.2 Role Of The Type System

A type system is the set of rules that a language uses to connect types with values and expressions. Some type systems are static, meaning they check most rules before the program runs. Others are dynamic, meaning they check many rules while the program is running. Many modern languages mix both ideas so that they can catch many errors early and still remain flexible.

For user defined types, the type system helps catch mismatches. If a function expects a Track and you pass a Patient, the compiler can stop you. If a field is declared as an enumeration and you try to store a number outside the allowed set, the system can complain. The type system is therefore a partner in design. When your types are well defined, the type system can enforce your design and protect the program from many common mistakes.

3.0 NATURE OR TYPE

3.1 Main Kinds Of User Defined Types

User defined types appear in many forms, depending on the language and the style of programming. The most common forms are records or structs, classes and objects, enumerations, and variants or unions. Each form has its own nature and is suited to certain modeling tasks. A good designer learns the strengths and limits of each kind.

When you design a system, it helps to know which form to choose. A record is simple and close to the data. A class adds behavior and hiding of details. An enumeration gives a short list of states. A variant lets a value take one of several shapes. Together, they form a small toolbox for modeling many different problems in a tidy way.

3.1.1 Records Or Structs

A record, often called a struct in languages like C, is a simple user defined type that groups several named fields. Each field has its own type. All fields are stored together in memory, often in a fixed layout. Records are good for modeling simple objects that mostly store data and do not need complex behavior. They are easy to understand and match well with tables or rows in a database.

For example, a record for a student might contain an integer field for the student id, a string field for the name, and another string field for the major. Functions that work on students can take a record as a parameter, but the record itself does not carry methods. This keeps the data and behavior separate and easy to see. A Track type in a low level language might also be written as a struct that holds fields for title, artist, and duration.

3.1.2 Classes And Objects

A class is a more powerful user defined type that supports both data and methods in one unit. An object is a concrete instance of a class. Classes are central to object oriented programming languages. They allow ideas like encapsulation, which means hiding inner details, and inheritance, which means sharing behavior between related classes.

When we say that user defined types are a means to model real systems, classes are often what we imagine. A class can hide its internal fields and show only a clean interface. This gives other parts of the program a simple way to use the model without needing to know how it is built inside. A Track class in an object oriented MP3 player might include methods such as play, pause, displayDetails, and addToPlaylist, so that all behavior related to tracks is bundled with the data.

3.1.3 Enumerations

An enumeration, or enum, is a user defined type that lists a small set of named values. It is useful when a variable can only take one value from a limited list, such as days of the week, levels of access, or states of a process. Enums make code safer and clearer because they avoid magic numbers and repeated string names that can be typed wrongly.

Fig. 2

Comparison of User-Defined Types

Characteristic	Records/Structs	Classes/Objects	Enumerations	Variants/Unions
Data Storage	Groups named fields	Data and methods in one	Lists named values	Value takes several shapes
Behavior	Mostly store data	Supports data and methods	Defines a limited list	Not explicitly mentioned
Use Case	Modeling simple objects	Modeling real systems	Representing limited states	Representing different shapes
Example	Student record with ID, name	Track class with play, pause	PlayerState enum with STOPPED	Not explicitly mentioned

This table compares user-defined types such as records, classes, enumerations, and variants. It explains how they differ in data storage, behavior, use cases, and examples, helping learners choose the right type for simple data, real-world objects, limited states, or multiple forms.

For example, instead of using 0, 1, and 2 to mean Stopped, Playing, and Paused, a music player program can define an enum called PlayerState with members STOPPED, PLAYING, and PAUSED. The compiler can then check that only these states are used in the code. Reading such code feels like reading short English phrases, which helps both correctness and understanding.

3.1.4 Variants And Unions

A variant or union type allows a value to take one of several different shapes. At any time, the value has one active form, and the program must know which form is in use. Modern languages often provide safe variant types, sometimes called tagged unions, which carry both the data and a tag that says which case is active at the moment.

Variants are helpful when modeling data that can appear in several related forms. A media app may have a MediaItem type that can be a Track, a Video, or a PodcastEpisode. Each case may use different fields, but all share some common behavior, such as “play” and “showDetails.” Because the cases are grouped into one variant type, the program can handle a mixed list of media items while still using the type system to keep track of which form each item has.

3.2 Mutable And Immutable Types

Another part of the nature of a type is whether its values can change after creation. A mutable type allows its fields to be updated. An immutable type does not allow such changes; instead, any update creates a new value. Immutable types are common in functional programming and can reduce bugs related to unexpected changes in shared data.

For user defined types, you should decide which fields, if any, may change. For example, a Patient type might allow the address to change over time, but it should not allow the id or date of birth to change. In an MP3 library, a Track type might be immutable, because the title and duration of a song rarely change. A Playlist, on the other hand, may be mutable, because users often add or remove tracks as their taste changes. Clear rules about mutability make programs easier to reason about.

4.0 EXAMPLES AND CASE STUDIES

4.1 Simple Examples

To see how user defined types help us model real situations, it is useful to start with a few simple examples. These examples use plain language rather than full code, but they show how types gather related information into one unit. After you understand these small cases, it becomes easier to read and design larger models.

4.1.1 Example Of A Student Type

Suppose we want to build a small system to track students in a course. If we try to use only primitive types, we might have separate arrays for student ids, names, and grades. This is hard to manage and easy to break, because a mistake in indexing can mix up one student with another.

Instead, we define a user defined type named Student with fields such as id, name, email, and grade. Now a single Student value keeps related data together. When we pass a Student to a function that prints a report or updates a grade, we know that all needed data travels as a unit. The Student type becomes a new word in the language of the program, and the code becomes easier to read.

4.1.2 Example Of A Mp3 Track Type

Now consider an MP3 music player. We can define a Track type with fields such as title, artist, album, durationInSeconds, and filePath. Each Track value stands for one song in the library. The filePath tells the player where to find the audio data. The other fields are used to show the user information about the song and to sort or search the library.

Because all of these facts are collected into one Track type, the program can work at a higher level. Instead of passing separate strings and numbers, it can pass a single Track value to functions or methods. This makes the code easier to read, lowers the chance of mixing up fields, and supports features like playlists and search filters in a natural way.

4.1.3 Example Of A Playlist Type

A Playlist type can hold a name and an ordered list of tracks. The list may be a simple array or a more advanced collection. The Playlist may also have behavior, such as methods to addTrack, removeTrack, and moveTrackUp. Many users think about their music in terms of playlists, such as “Study Songs” or “Workout Mix,” so modeling playlists as user defined types is natural and useful.

Internally, each playlist holds references to Track values. The playlist does not copy the audio files; it simply keeps pointers to tracks that already exist in the library. This design allows one

song to appear in many playlists without taking extra space. It also shows how different types, such as Playlist and Track, can cooperate to model a richer system.

4.2 Case Study Of A Mp3 Music Library

4.2.1 Data Types In The Library

In a small MP3 music library system, we might have several core types: Track, Artist, Album, Playlist, and MediaLibrary. The MediaLibrary type can hold lists of all tracks, artists, and playlists. The Artist type includes the artist name and a list of tracks by that artist. The Album type includes the album title, year, and a list of tracks in album order. The Playlist type holds user-chosen groups of tracks.

Each of these types mirrors something in the user's mental picture of their music collection. People think about songs, artists, albums, and playlists, so the program should do the same. By designing these types carefully, we make later tasks, such as searching for a song by artist, listing all albums in a year, or creating a new playlist, much easier to implement and to understand.

4.2.2 Operations On The Library

Once the types are in place, many operations become simple. To find all tracks by a given artist, the program can search the list of Artist values and use the list of tracks stored in the matching artist. To play a playlist, it can go through its list of Track values, one by one, and send each track to the player. To remove an album, it can remove its tracks from the library lists, using the links between Album and Track.

These operations are easier to think about because the user defined types match real concepts. The program talks in terms of tracks, artists, albums, and playlists, just as the user does. If we had tried to build the same system with only primitive types and loose arrays, the logic would be much harder to follow and far easier to break. The MP3 library case study shows clearly how user defined types are a means to model and organize complex data.

4.3 Case Study Of A Hospital System

4.3.1 Patient, Doctor, And Appointment Types

Our second case study is a small hospital information system. Here, the key actors are patients, doctors, and appointments. User defined types help us keep each of these ideas clear and separate while also linking them in the right way.

We can define a Patient type that stores id, name, dateOfBirth, bloodGroup, and address. A Doctor type can have id, name, specialty, and a list of assigned patients. An Appointment type

connects one patient, one doctor, a date and time, and a status field that uses an enumeration such as SCHEDULED, COMPLETED, or CANCELLED.

By tying these types together, we get a rich model of the system. When a new appointment is created, the program creates an Appointment value that links to one Patient and one Doctor. When we print a schedule, we loop over Appointment values rather than over separate lists of names and times. The program can now speak in terms of Patients, Doctors, and Appointments, just as the hospital staff do in real life.

4.4 Common Mistakes And Good Practices

4.4.1 Typical Errors In Modeling

Beginners often make a few common mistakes when designing user defined types. One mistake is to use only primitive types, even when a new type would make the code clearer. For example, they may keep many parallel arrays instead of creating a single Student or Track type. Another mistake is the opposite: creating too many tiny types that do not add meaning, such as wrapping an integer id in a separate type that is never given more structure or behavior.

A further mistake is to mix unrelated ideas in one type. For example, a Student type that also stores details about the last exam room and the cafeteria balance may be carrying too much. In such cases, it is better to create smaller types, such as ExamRecord and PaymentAccount, and link them to the Student type. Mixing too many roles in one type makes the code hard to read and hard to change safely.

4.4.2 Guidelines For Better Designs

Good modeling tries to give each type one main purpose. Names should be clear, fields should match real facts, and relationships between types should reflect real relationships in the world. In a music system, Track, Artist, Album, and Playlist should each have their own clear roles. In a hospital system, Patient, Doctor, and Appointment should also have distinct and well understood roles.

When in doubt, ask yourself what nouns appear in the problem story. These nouns are good candidates for user defined types. Then ask what facts belong to each noun and what actions should be done with it. Over time, this simple habit will make your designs stronger and your code easier to understand. Well chosen user defined types will guide both the compiler and future programmers who read your work.

1. User defined types help programmers model real world ideas inside programs in a clear way.
2. Primitive types act as basic building blocks, while user defined types build richer models on top of them.

3. Records, classes, enumerations, and variants are common forms of user defined types with different strengths.
4. Good models keep each type focused on one main purpose and match real nouns in the problem story.
5. An MP3 library can be modeled with Track, Artist, Album, Playlist, and MediaLibrary types working together.
6. A hospital system can be modeled with Patient, Doctor, and Appointment types linked by clear relationships.
7. The type system supports these designs by catching mismatches and unsafe uses of values.
8. Thinking formally about types helps prove that code is correct and avoids many common errors.

5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple Meaning In This Unit
Data type	Rule that says what kind of value we have and how we can use it
Primitive type	Basic built in type such as integer, float, string, or Boolean
User defined type	New type created by a programmer for a specific problem
Modeling	Building a simple picture of real things inside a program
Record / struct	Group of named fields stored together as one value
Class	Type that holds data and methods in one unit
Object	One actual instance created from a class
Enumeration (enum)	Type with a small fixed set of named states
Variant / union	Type whose value can take one of several different shapes
Track	Type that models one MP3 song with fields like title and artist
Playlist	Type that stores a named, ordered list of tracks
Patient	Type that models a person in a hospital, with id and health data
Appointment	Type that links patient, doctor, time, and status
Mutable	Able to change after creation by updating fields
Immutable	Cannot change once created; changes make a new value